

Architecture Microservices — Fiche de soutenance

Projet Motus · le Wordle français en microservices · M2 MIAGE SITN — Dauphine

BINÔME

Liya & Steven

ENSEIGNANT

M. Mouloud Menceur

SOUTENANCE

Mar. 7 juillet 2026

STACK

Java 26 · Spring Boot 4.0.7 · MySQL · Docker

Cette fiche couvre les 4 parties du cours (Web-scale IT, Monolithe→Microservices, Spring Boot, REST), le projet (ce qu'on a fait, comment, pourquoi), la conformité au cours, et le plan Minikube (pas encore fait). Les notions sont mises en avant car la soutenance portera surtout dessus.

I De l'Enterprise-scale IT au Web-scale IT

Web-scale IT

Façon de concevoir l'IT des géants du web (Google, Amazon, Netflix) : systèmes qui doivent tenir une charge énorme, évoluer vite et **ne jamais tomber en panne totale**. On abandonne le gros serveur unique au profit de nombreux petits services distribués.

Cloud · Virtualisation · Conteneurisation

- **Cloud** : ressources (calcul, stockage) à la demande, payées à l'usage (IaaS/PaaS/SaaS).
- **Virtualisation** : plusieurs machines virtuelles (VM) sur un serveur physique (hyperviseur).
- **Conteneurisation** : plus léger qu'une VM, partage le noyau de l'OS → **Docker**. Base de nos microservices.

DevOps · IaC · GitOps · Continuous Delivery

- **DevOps** : rapprocher Dev + Ops, automatiser build/test/déploiement (CI/CD).
- **IaC** (Infrastructure as Code) : décrire l'infra dans des fichiers versionnés (ex. nos `docker-compose.yml`, futurs manifests K8s).
- **GitOps** : Git = source de vérité de l'infra et du déploiement.
- **Continuous Delivery** : livrer souvent, par petits incréments.

Organisation & qualité

- **Feature teams / équipes 2-pizzas** : petites équipes pluridisciplinaires, autonomes, alignées « produit ».
- **Software Craftsmanship** : **SOLID**, Clean Code, tests. Priorité au code de qualité, maintenable.

Scalabilité & élasticité

- **Verticale** : grossir une machine (limité).
- **Horizontale** : ajouter des instances → **load balancing** réparti la charge.
- **Auto-scaling / élasticité** : ajout/retrait automatique d'instances selon la charge (rôle de Kubernetes).

Design for failure · Chaos Engineering

On **conçoit en supposant que ça va tomber** : redondance, isolation des pannes, redémarrage. **Chaos Engineering** (ex. Netflix « Chaos Monkey ») : casser volontairement des composants en prod pour vérifier la résilience.

Pourquoi ça compte pour Motus : nos 4 services sont petits, indépendants, conteneurisables et redémarrables séparément — exactement l'esprit Web-scale IT. Si `score-service` tombe, on peut encore jouer une partie (le jeu marche, seul le classement attend).

II Du monolithe aux microservices

Architecture en couches (monolithe Java EE)

Présentation (UI) → **Services** (cas d'usage / API) → **Logique métier** → **DAO** (accès aux données). Tout est empaqueté dans **une seule application** déployée sur un serveur d'applications.

Limites du monolithe

- Gros projet complexe, difficile à faire évoluer et maintenir.
- Scalabilité « tout ou rien » (on duplique toute l'appli).
- Un bug/redémarrage impacte toute l'application.

SOA & ESB

SOA : organiser le SI en services réutilisables communiquant via un protocole standard (**SOAP** = HTTP+XML). **ESB** : bus central d'intégration (routage, transformation). Problème : **architecture centralisée** → **SPOF**, lourde, scalabilité limitée.

Microservices — définition (M. Fowler)

« Développer une application comme **une suite de petits services**, chacun dans **son propre processus**, communiquant par des mécanismes légers, souvent une **API HTTP**. »

Caractéristiques d'un microservice

Autonome & petit — collabore avec les autres mais vit seul.

Single Responsibility — une seule fonctionnalité métier.

Base de données propre — l'autonomie va jusqu'aux données.

Déployable indépendamment — mis à jour/remplacé sans toucher aux autres.

Communication simple — HTTP / REST (pas d'ESB).

Cloud-native — frameworks légers (Spring Boot), conteneurs.

SOA traditionnelle	Microservices
Maximise la réutilisabilité	Maximise le découplage
Service « gros grain » (responsabilités multiples)	Petit service, une responsabilité
ESB, protocoles complexes (SOAP)	Communication directe (HTTP, REST)
Une seule base pour tout le monolithe	Une base par service
Gouvernance commune imposée	Liberté technologique par équipe

API Gateway

Point d'entrée unique devant les microservices : route les appels, centralise l'authentification, le logging, la limitation de débit. Simplifie la vie du client (une seule adresse).

Observabilité (3 piliers)

- **Logs** : événements détaillés de chaque service (souvent centralisés).
 - **Traces distribuées** : suivre une requête **de bout en bout** à travers les services → **OpenTelemetry**.
 - **Métriques** : compteurs/jauges (latence, débit) → **Prometheus + Grafana**.
- « Sans observabilité, pas de surveillance possible. »

Inconvénients (à assumer en soutenance) : système distribué = plus de trafic réseau, de latence, d'éléments à surveiller, organisation plus complexe. « Il faut que l'application s'y prête. » Pour un jeu découpé en player/game/dict/score, ça s'y prête bien.

III Développement avec Spring Boot

IoC & Injection de dépendances

Inversion de Contrôle (IoC) : c'est le conteneur **Spring** qui crée et orchestre les objets, pas nous. **Injection de dépendances** : Spring fournit à un objet ses dépendances (ex. le repository injecté dans le controller). Base du principe **SOLID** (D = Dependency Inversion).

Maven

Outil de **build** Java (« convention over configuration ») : arborescence standard, cycle de vie (compile → test → package), **gestion transitive des dépendances**. Config dans `pom.xml` (POM = Project Object Model).

Spring → Spring Boot

Spring = framework modulaire mais **config fastidieuse**. Spring Boot = Spring **préconfiguré** (« convention over configuration ») :

- App autonome, **serveur Tomcat embarqué** (`java -jar`).
- **Starters** Maven (regroupent les dépendances).
- **Actuator** : supervision en production.
- `start.spring.io` pour générer le projet.

@SpringBootApplication

Classe générée = **point d'entrée** ; l'annotation déclenche l'**auto-configuration**. Paramètres externalisés dans `application.properties` (port, datasource...).

Persistance : JPA · Hibernate · Spring Data ★ (le point ORM du cours)

Les 3 mots à ne pas confondre

- **ORM** (Object-Relational Mapping) : **principe** — faire correspondre un objet Java ↔ une ligne de table SQL, sans écrire le SQL à la main.
- **JPA** : la **norme** Java de l'ORM (annotations `@Entity`, `@Id` ...).
- **Hibernate** : l'**implémentation** de JPA utilisée par défaut par Spring Boot (le moteur qui génère le SQL).
- **Spring Data JPA** : couche au-dessus qui **génère les requêtes** à partir du nom des méthodes (aucun DAO à écrire).

Base de données

H2 (cours) : SQL en mémoire, ultra-léger, auto-configuré — **idéal pour les tests** (données perdues à l'arrêt). **MySQL** (notre choix) : base réelle, persistante. `data.sql` dans `resources` = script exécuté au démarrage (insertion des données).

Comment on l'utilise (cf. cours TauxChange)

- **Entité** : classe annotée `@Entity` + `@Id @GeneratedValue` + `@Column`.
- **Repository** : interface `extends JpaRepository<Entité, Long>` → CRUD gratuit (`save`, `findAll` ...).
- **Query methods** : `findBySourceAndDest(...)` → Spring déduit le SQL du nom.
- `@Query` : requête JPQL ou SQL natif quand on veut la maîtriser.

Le contrôleur REST

`@RestController` : la classe traite les requêtes et renvoie directement la réponse (JSON). `@GetMapping` / `@PostMapping` mappent une méthode à une URL. `@PathVariable` / `@RequestParam` / `@RequestBody` lisent l'URL et le corps. `@Autowired` / injection par constructeur pour les dépendances.

IV REST — REpresentational State Transfer

REST vs SOAP

SOAP (SOA des années 2000) : RPC = HTTP + XML, lourd, complexe. **REST** (thèse de Roy Fielding, 2000) : style d'architecture **simple** basé sur HTTP. Aujourd'hui standard des API web (JSON).

Rappels HTTP

Verbes : **GET** (lire, idempotent), **POST** (créer), **PUT** (remplacer), **DELETE** (supprimer). Codes : **2xx** succès (200 OK, 201 Created), **4xx** erreur client (401, 403, **404**, 409 Conflict), **5xx** erreur serveur (500, 503).

Les contraintes REST

1. Ressources identifiées par une **URI** (`/words/random`).

2. Interface uniforme : verbes + codes HTTP standard.

3. Stateless : aucune session serveur → **scalabilité horizontale** + résilience.

4. Cache HTTP : éviter de requêter systématiquement l'origine.

5. Système en couches : le client ignore s'il parle à un intermédiaire (cache, gateway, load-balancer).

Représentation par type de média : `application/json`.

Modèle de maturité de Richardson

Niveau	Idée	Caractéristique
0 — RPC sur HTTP	« The Swamp of POX »	Une seule URI, tout en POST, XML, code 200 même en erreur (≈ SOAP).
1 — Ressources	Plusieurs URI	Chaque ressource a son URI, mais toujours POST partout.
2 — Verbes + codes 🌟	« Pragmatic REST »	On exploite GET/POST/PUT/DELETE + les codes HTTP (201, 409...). Niveau visé en pratique.
3 — HATEOAS	« The glory of REST »	La réponse contient des hyperliens vers les actions possibles → API auto-découvrable.

À retenir pour la soutenance : notre API est au **niveau 2** de Richardson (URI par ressource + bons verbes + bons codes HTTP), le niveau recommandé en pratique. HATEOAS (niveau 3) est puissant mais souvent jugé trop complexe pour le besoin.



Le projet Motus — ce qu'on a fait, comment, pourquoi

4 microservices Spring Boot + 1 frontend, chacun sa base MySQL, communication REST.

Vue d'ensemble — 4 microservices (Single Responsibility)

Service	Rôle (une responsabilité)	Port	Base MySQL	Qui
player-service	Joueurs : inscription, identité	8081	motus_players	Steven
game-service	Parties : algo Motus, essais, statut	8082	motus_games	Steven
dict-service	Dictionnaire : mot aléatoire, validation	8083	motus_dictionary	Liya
score-service	Scores : calcul, classement, stats	8084	motus_scores	Liya
frontend	UI (HTML/CSS/JS), thème Motus	—	—	Liya

Le chemin d'une partie (le flux complet à raconter au jury)

1. **Front** → POST /players (player-service) crée / retrouve le joueur
2. **Front** → POST /games (game-service) démarre une partie
 - ↳ game-service → GET /words/random?length=6 (**dict-service**) → mot mystère
3. **Front** → POST /games/{id}/guess chaque essai
 - ↳ MotusEngine compare (bien placé / mal placé / absent)
 - ↳ (option) POST /words/validate (**dict-service**) → mot autorisé ?
4. Partie gagnée/perdue → game-service **PUSH** POST /scores/results (**score-service**)
 - ↳ score-service calcule le score, persiste, met à jour le classement
5. **Front** → GET /scores/ranking, /scores/players/{id} ... classement & stats

ORM / JPA dans le projet

Chaque service a ses **entités** (`Mot`, `GameResult`, `Player`, `Game`) annotées `@Entity`, et ses **repositories** `JpaRepository`. Hibernate génère le SQL et crée les tables (`ddl-auto`). Ex. dict :

- `findByLongueur(int)` — query method
- `existsByMotIgnoreCase(String)` — validation
- `@Query(... ORDER BY RAND() LIMIT 1)` — tirage efficace sur 132k mots

REST dans le projet (Richardson niveau 2)

- GET /words/random?length=6 → {"word":"MAISON"}
- POST /words/validate → {"valid":true}
- GET /words/lengths → niveaux dispo
- POST /scores/results (push) · GET /scores/ranking

Bons verbes, bonnes URI par ressource, codes HTTP (200/201/404 via `GlobalExceptionHandler`), JSON camelCase.

Communication inter-services

RestTemplate (vu en cours) : game-service appelle dict-service (`DictServiceClient`) et score-service (`ScoreServiceClient`).
Modèle « **push** » : c'est game-service qui envoie le résultat à score-service en fin de partie (score-service ne connaît pas game-service → couplage faible).

Une base par service + CORS + tests

- **1 base MySQL par service** (`createDatabaseIfNotExists`) → autonomie des données.
- **CORS** (`WebMvcConfigurer`) : le front (autre origine) peut appeler les API.
- **Tests** : JUnit + H2 en mémoire (dict 18, score 28...), `@SpringBootTest`.

Idempotence (bon réflexe microservices) : `game_id` est **unique** dans score-service. Si game-service renvoie deux fois le même résultat (réseau), on ne compte pas le score en double.

Conformité au cours — « on a bien tout utilisé »

Notion du cours	Utilisée ?	Où / comment dans Motus
Architecture microservices (Fowler)	✓	4 services autonomes, 1 responsabilité, 1 base chacun
Spring Boot (auto-config, Tomcat embarqué)	✓	4 apps <code>@SpringBootApplication</code> , <code>java -jar</code>
Maven (pom.xml, starters)	✓	4 <code>pom.xml</code> , parent Spring Boot 4.0.7, Java 26
IoC / Injection de dépendances	✓	Injection par constructeur partout
ORM / JPA / Hibernate	✓	<code>@Entity</code> , <code>JpaRepository</code> , <code>ddl-auto</code> , dialecte Hibernate
Spring Data (query methods, @Query)	✓	<code>findByLongueur</code> , <code>@Query ORDER BY RAND()</code>
data.sql au démarrage	✓	132k mots chargés dans dict-service
H2 (tests)	✓	Scope test dans les 4 poms
REST niveau 2 (Richardson)	✓	URI/verbes/codes HTTP, JSON
RestTemplate (appel inter-services)	✓	Dict/Score ServiceClient
Actuator (supervision)	✓	<code>/actuator/health</code> exposé
Conteneurisation (Docker)	✓	<code>docker-compose.yml</code> : 4 services + MySQL
Scalabilité / auto-scaling (K8s)	🟡	Minikube = prochaine étape (voir ci-dessous)
Observabilité (traces, Prometheus/Grafana)	🟡	Logs + Actuator présents ; traces distribuées = amélioration possible

K8s Le plan Minikube / Kubernetes (à faire)

C'est quoi ?

Kubernetes (K8s) : orchestrateur de conteneurs — il déploie, surveille, redémarre et **scale automatiquement** les services sur un cluster. **Minikube** : un cluster K8s à **un seul nœud**, sur notre PC, pour tester en local. C'est le lien direct avec le cours (« ajout automatique de nœuds K8s », scalabilité, design for failure).

Pourquoi (lien cours)

- **Scalabilité horizontale** : `replicas: 3` → 3 instances d'un service.
- **Auto-scaling** : `HorizontalPodAutoscaler` monte les instances si CPU élevé.
- **Design for failure** : si un pod meurt, K8s en relance un (self-healing).
- **Service discovery** : les services se trouvent par leur nom DNS interne.

Les étapes concrètes qu'on va suivre

1 · Dockerfile par service

Aujourd'hui on a `docker-compose`. On écrit un `Dockerfile` par service (base `eclipse-temurin:26`, copie du `.jar`, `ENTRYPOINT java -jar`) puis `docker build` → 4 images.

2 · Démarrer Minikube

`minikube start` (crée le cluster local). `eval $(minikube docker-env)` pour que le cluster voie nos images locales.

3 · Déployer MySQL

Un `Deployment` + `Service` MySQL (+ un `PersistentVolumeClaim` pour garder les données, et un `Secret` pour le mot de passe).

4 · Un Deployment + Service par microservice

Pour chacun (player/game/dict/score) : un `Deployment` (image + replicas + variables d'env comme `SPRING_DATASOURCE_URL`) et un `Service` (nom DNS interne, ex. `dict-service:8083`).

5 · Exposer le front

Un `Service` type `NodePort` (ou un `Ingress` jouant le rôle d'**API Gateway**) pour accéder à l'appli depuis le navigateur.

6 · Appliquer & vérifier

`kubectl apply -f k8s/` puis `kubectl get pods`, `minikube service front`. Démo : supprimer un pod → K8s le relance (design for failure).

```
k8s/
├─ mysql-deployment.yaml + mysql-service.yaml + mysql-pvc.yaml
├─ player-deployment.yaml + player-service.yaml
├─ game-deployment.yaml   + game-service.yaml
├─ dict-deployment.yaml   + dict-service.yaml
├─ score-deployment.yaml  + score-service.yaml
└─ frontend-deployment.yaml + frontend-service.yaml (NodePort / Ingress)
```

Phrase de synthèse pour le jury : « docker-compose orchestre nos conteneurs en local ; Minikube fait la même chose avec Kubernetes, en ajoutant le scaling automatique, l'auto-réparation et la découverte de services — ce qui rend l'architecture réellement *cloud-native*. »

Questions-types de soutenance (préparées)

Q. Différence monolithe / microservices ?

Monolithe = une appli en couches, déployée d'un bloc, une base. Microservices = petits services autonomes, une responsabilité et une base chacun, déployables séparément, communiquant en HTTP/REST.

Q. Qu'est-ce qu'un ORM ? JPA vs Hibernate ?

ORM = mapper objets Java ↔ tables SQL. JPA = la norme (annotations). Hibernate = l'implémentation qui génère le SQL. Spring Data JPA génère les requêtes depuis le nom des méthodes.

Q. Pourquoi une base par service ?

Autonomie : chaque service évolue et se déploie sans casser les autres ; pas de couplage par la base. C'est une caractéristique clé du cours.

Q. À quel niveau REST êtes-vous ?

Niveau 2 de Richardson : ressources par URI, verbes HTTP, codes de statut. HATEOAS (niv. 3) non retenu car complexe pour le besoin.

Q. Comment communiquent vos services ?

En REST via RestTemplate. game-service appelle dict-service (mot/validation) et pousse le résultat à score-service (modèle push, couplage faible, idempotent par game_id).

Q. Que se passe-t-il si un service tombe ?

Isolation des pannes : le jeu marche sans score-service (le classement attend). Avec K8s/Minikube, un pod mort est relancé automatiquement (design for failure).

Rappel des choix à défendre : (1) **MySQL** au lieu de H2 → persistance + « une base par service » ; H2 gardé pour les tests. (2) **Modèle push** game→score → couplage faible. (3) **Java 26 / Spring Boot 4.0.7** → accord binôme, versions récentes. (4) **Docker-compose** aujourd'hui, **Minikube** demain.